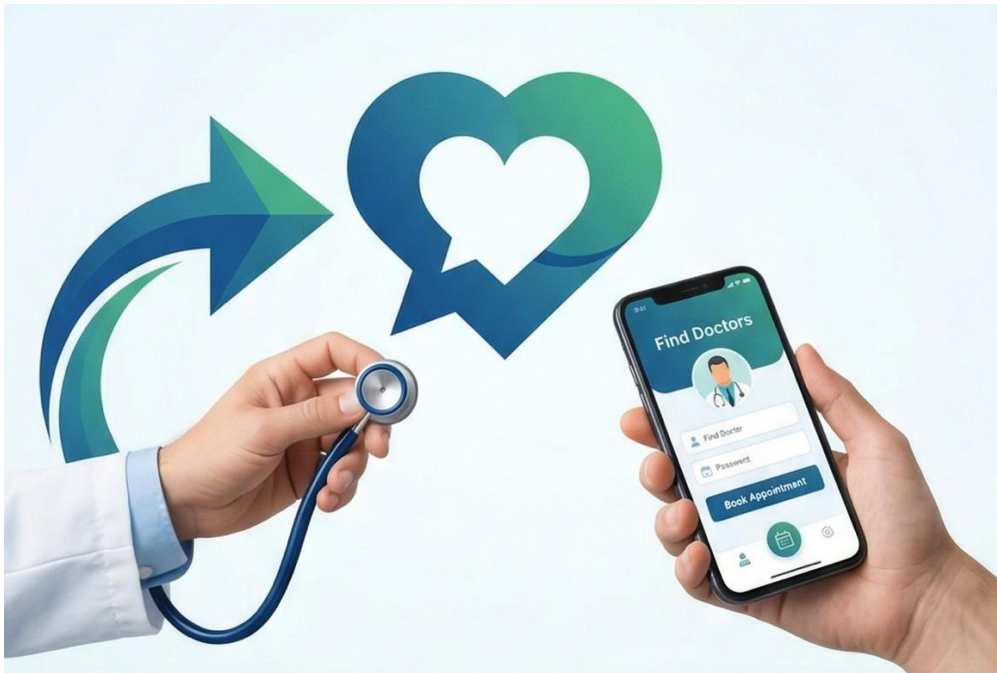




Medic-Go Telemedicine Platform

IT Software Solutions for Business

LKS Provinsi

2026

Technical Project Specification

1. Project Strategic Overview

Medic-Go is a centralized telemedicine platform engineered to bridge the gap between patients and healthcare providers through a seamless digital interface. In the current landscape, digitizing healthcare access is a strategic necessity to reduce friction in patient-provider interactions and ensure timely medical interventions. This specification focuses on the **"Patient-First"** development phase, prioritizing the features and infrastructure necessary for patients to discover medical expertise and manage their health journey efficiently.

The primary mission of Medic-Go is to streamline the medical appointment booking process. By providing a secure, high-performance environment, the platform ensures that sensitive healthcare data is handled with maximum integrity while offering an intuitive navigation experience for users.

To achieve a comprehensive full-stack assessment, the project is divided into two distinct development tracks:

- A robust **REST API (API_XX)** for backend services
- A high-fidelity **Mobile Client (MedicXX)** for patients

2. Phase 1: REST API Architecture (Medic-Go Backend)

The backend architecture focuses on security and transactional integrity. By implementing **Role-Based Access Control (RBAC)**, the system ensures data confidentiality.

- **Base URL:** `https://<host>/medicgo-api/v1`
- **Project Name:** `API_XX`

2.1 Authentication Endpoints

2.1.1 User Registration

- **Method:** `POST /users/register`
- **Hashing Implementation:** Passwords must be hashed using **SHA256**.
 - Logic: `UTF8 Encoding -> ComputeHash -> Convert to Hex string`
- **Validation:**
 - Password: Min 8 chars, must include Uppercase, Lowercase, Symbol, and Number.
 - Email: Must be unique and properly formatted.
 - Phone: Must be numeric only.

Request:

```
{
  "username": "patient_01",
  "fullName": "John Doe",
  "email": "john@example.com",
  "phone": "08123456789",
  "password": "SecurePass123!"
}
```

Success Response (200):

```
{
  "message": "User registered successfully."
}
```

2.1.2 User Login

- **Method:** `POST /users/login`
- **JWT Logic:** Returns a token expiring in **7 days** embedding the user's role.

Success Response (200):

```
{
  "message": "Login successful.",
  "data": {
    "userId": 1,
    "username": "patient_01",
    "role": "patient",
    "token": "eyJhbGciOiJIUzI1Ni..."
  }
}
```

2.1.3 Logout

- **Method:** POST /users/logout
- **Requirement:** Invalidate the JWT token. Requires Authorization: Bearer <token>.

2.1.4 Get Current User Profile

- **Method:** GET /users/profile
- **Requirement:** Requires Authorization: Bearer <token>.

Success Response (200):

```
{
  "data": {
    "userId": 1,
    "username": "patient_01",
    "fullName": "John Doe",
    "email": "john@example.com",
    "phone": "08123456789",
    "role": "patient"
  }
}
```

2.2 Appointment & Provider Management

2.2.1 Get Doctor Listings

- **Method:** GET /doctors
- **Logic:** Filter by specialty (partial match). Default sort: created_at Descending.
- **Pagination:** page (default 1), size (default 10).

Success Response (200):

```
{
  "data": [
    {
      "id": 1,
      "name": "Dr. Smith",
      "specialty": "Cardiology",
      "experience": "Min. 8 years",
      "location": "Jakarta Selatan",
      "price": 150.00
    }
  ],
  "pagination": {
    "page": 1,
    "size": 10,
    "totalPages": 5
  }
}
```

2.2.2 Get Doctor Detail

- **Method:** GET /doctors/{id}
- **Requirement:** Returns full doctor profile including the expertise list.

Success Response (200):

```
{
  "data": {
    "id": 1,
    "name": "Dr. Smith",
    "specialty": "Cardiology",
    "description": "Dr. Smith is a specialist in cardiovascular diseases...",
    "experience": "Min. 8 years",
    "location": "Jakarta Selatan",
    "price": 150.00,
    "duration": 30,
    "expertise": [
      { "title": "Echocardiography", "content": "Non-invasive cardiac imaging." },
      { "title": "Interventional Cardiology", "content": "Catheter-based procedures." }
    ]
  }
}
```

2.2.3 Booking an Appointment

- **Method:** POST /appointments
- **Validation:** paymentMethod must be debit_card, credit_card, or paypal. If a coupon is applied, verify expiry_date > today and quota > 0.

Request:

```
{
  "doctorId": 1,
  "paymentMethod": "credit_card",
  "couponCode": "HEALTH20"
}
```

2.2.4 Get Patient Appointments

- **Method:** GET /appointments
- **Requirement:** Requires Authorization: Bearer <token>. Returns all appointments for the authenticated patient.
- **Logic:** Default sort: created_at Descending.

Success Response (200):

```
{
  "data": [
    {
      "id": 1,
      "doctorId": 1,
      "doctorName": "Dr. Smith",
      "specialty": "Cardiology",
      "paymentMethod": "credit_card",
      "status": "Waiting for Confirmation",
      "createdAt": "2026-03-23T10:00:00Z"
    }
  ]
}
```

2.3 Saved Doctors

2.3.1 Save a Doctor

- **Method:** POST /saved-doctors
- **Requirement:** Requires Authorization: Bearer <token>.

Request:

```
{
  "doctorId": 1
}
```

Success Response (200):

```
{
  "message": "Doctor saved successfully."
}
```

2.3.2 Get Saved Doctors

- **Method:** GET /saved-doctors
- **Requirement:** Requires Authorization: Bearer <token>.

Success Response (200):

```
{
  "data": [
    {
      "savedId": 1,
      "doctorId": 1,
      "doctorName": "Dr. Smith",
      "specialty": "Cardiology",
      "experience": "Min. 8 years",
      "location": "Jakarta Selatan"
    }
  ]
}
```

2.3.3 Remove a Saved Doctor

- **Method:** DELETE /saved-doctors/{id}
- **Requirement:** Requires Authorization: Bearer <token>.

Success Response (200):

```
{
  "message": "Saved doctor removed successfully."
}
```

2.4 Admin Operations (RBAC: Admin Only)

- **Add Doctor Profile:** `POST /doctors`
 - **Requirement:** Must include at least **3** "Expertise/Service" modules.
 - **Validation:** `price` and `duration` must be `> 0`.
- **Error Handling:** Non-admin attempts must return `403 Forbidden` with message: `"Access denied. Admin role required."`

2.5 Standardized Error Codes

Code	Status	Description
401	Unauthorized	Missing/invalid token or credentials.
403	Forbidden	Role mismatch (e.g., patient accessing admin tools).
422	Validation Error	Logic violation (e.g., quota must be a positive integer).

3. Phase 2: Mobile Application Development (Patient Client)

The User Experience (UX) strategy for the Medic-Go mobile application is centered on clarity, trust, and accessibility. Health-related interfaces must minimize cognitive load, ensuring that patients who may be in distress can find doctors and manage appointments without unnecessary complexity. The visual language utilizes a professional aesthetic to foster reliability.

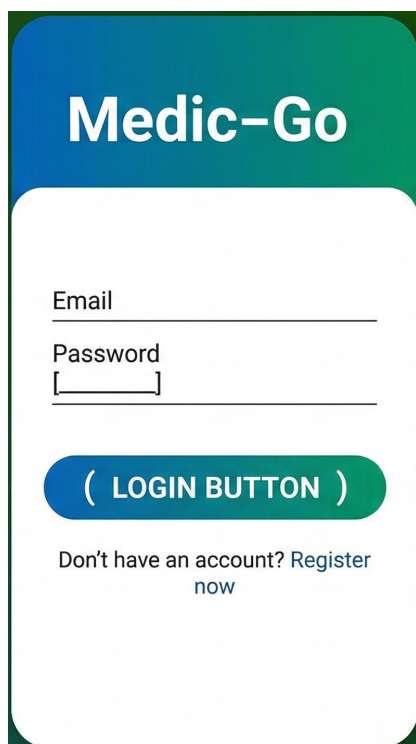
3.1 Technical Constraints

- **Project Naming:** The project must be named `MedicXX` (where `XX` represents your assigned PC Number).
- **Dependencies:** Developers are strictly forbidden from adding external libraries or third-party dependencies beyond the provided boilerplate code.

3.2 Functional Requirements & UI Mockups

3.2.1 Login Screen

The entry point for patients to authenticate and access their medical dashboard.



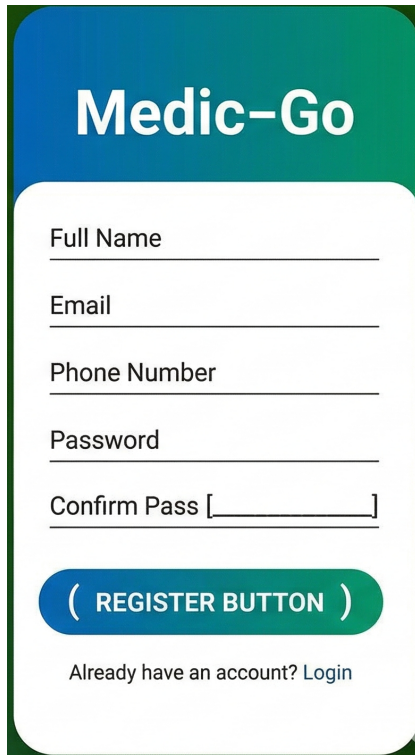
The mockup shows a mobile application login screen. At the top is a green header with the text "Medic-Go" in white. Below the header is a white rounded rectangle containing the login form. The form has two input fields: "Email" and "Password". The "Password" field has a small square icon for toggling visibility. Below the input fields is a green rounded button with the text "(LOGIN BUTTON)". At the bottom of the white rounded rectangle, there is a link that says "Don't have an account? Register now".

Requirements:

- Only "**Patient**" roles can access the mobile client.
- Navigation link for new patients to access the Registration screen.

3.2.2 Register Screen

Enables new patients to create a secure account.



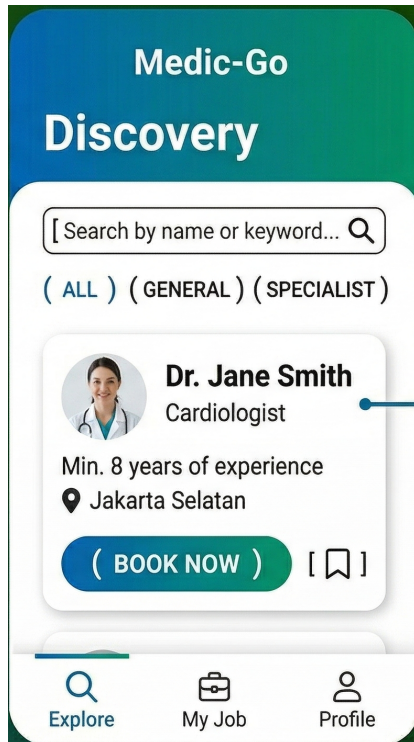
The image shows a mobile app registration screen for 'Medic-Go'. The header has a blue-to-green gradient with the app name 'Medic-Go' in white. Below the header is a white rounded rectangle containing the registration form. The form has five input fields: 'Full Name', 'Email', 'Phone Number', 'Password', and 'Confirm Pass' (with a trailing bracket). Below these fields is a green rounded button with the text '(REGISTER BUTTON)'. At the bottom of the white box, there is a link that says 'Already have an account? Login'.

Requirements:

- **Fields:** Full Name, Email, Phone Number, Password, and Confirm Password.
- **Email Validation:** Must be unique and follow standard formatting.
- **Phone Number:** Must be restricted to numeric input only.
- **Password Integrity:** Confirmation password must match the primary password field.

3.2.3 Doctor Discovery (Home) Screen

The primary hub for finding healthcare providers.

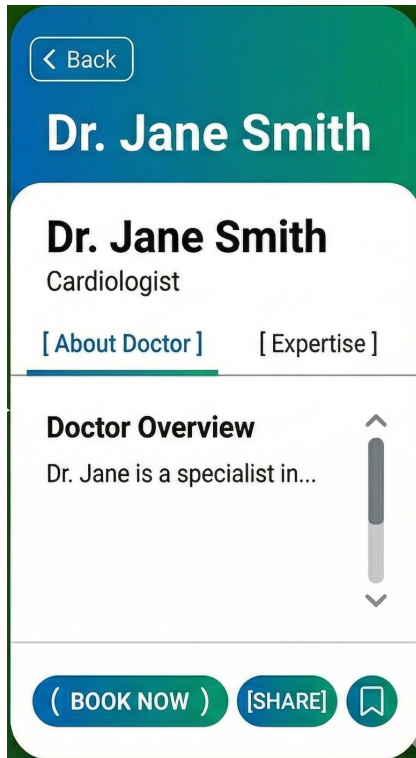


Requirements:

- **Search:** Keyword-based search with partial match functionality.
- **Filtering:** Category filters for "All", "General Practitioner", and "Specialist".
- **Doctor Cards:** Display Name, Specialty, Experience (e.g., "Min 5 years"), and Location.
- **Persistence:** Ability to "Save" a doctor profile for later.

3.2.4 Doctor Detail Screen

In-depth profile view utilizing a tabbed interface for clarity.

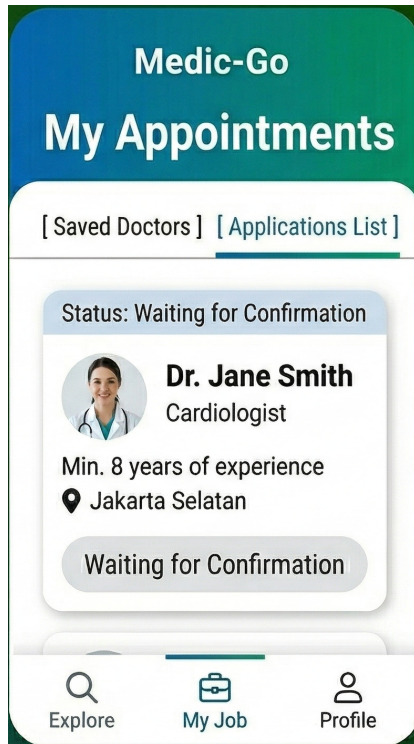


Requirements:

- **Tabs:** "About Doctor" (Overview/Bio) and "Expertise" (List of services/specializations).
- **Share Feature:** Action button to share text: "Check out this doctor: [doctor_link]".
- **Booking:** Primary action to navigate to the appointment booking flow.

3.2.5 My Appointments Screen

Management interface for tracking medical consultations.

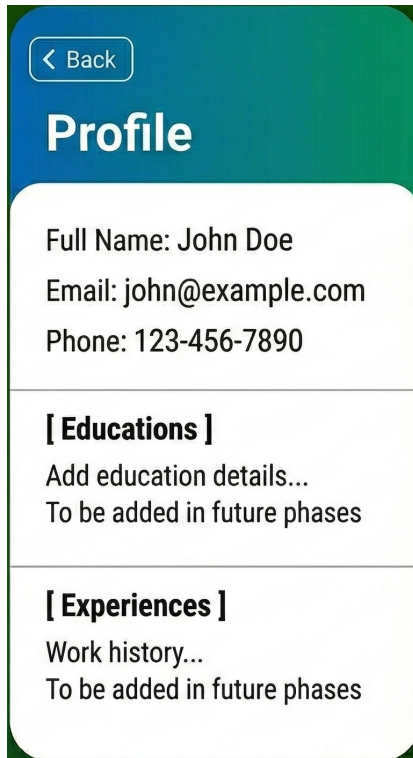


Requirements:

- **Tabs:** "Saved Doctors" (Removeable list) and "Applications List" (Status tracking).
- **Status Indicators:** Display clear status like "Waiting for Confirmation".

3.2.6 Profile Screen

Centralized view of the patient's personal data.



The image shows a mobile app profile screen. At the top is a blue header with a white '< Back' button and the title 'Profile' in white. Below the header is a white card with rounded corners. The card contains three sections: 1) Personal Information: 'Full Name: John Doe', 'Email: john@example.com', and 'Phone: 123-456-7890'. 2) Educations: A section header '[Educations]' followed by the text 'Add education details...' and 'To be added in future phases'. 3) Experiences: A section header '[Experiences]' followed by the text 'Work history...' and 'To be added in future phases'.

< Back

Profile

Full Name: John Doe
Email: john@example.com
Phone: 123-456-7890

[Educations]

Add education details...
To be added in future phases

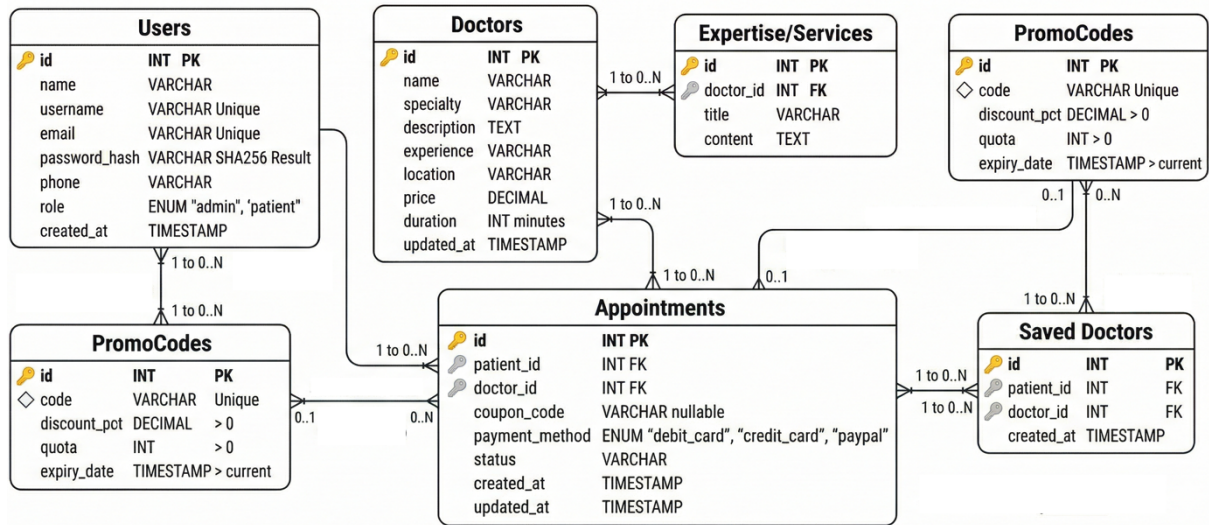
[Experiences]

Work history...
To be added in future phases

Requirements:

- Display Name, Email, and Phone.
- Placeholders for "Educations" and "Experiences" to be added in future phases.

4. Technical Data Schema



5. Deliverables & Implementation

Final submission must include:

1. **Mobile Client:** `MedicXX` source code with all 6 screens functional.
2. **REST API:** `API_XX` source code.
3. **Documentation:** Swagger UI accessible at `http://localhost:5000/swagger`.